

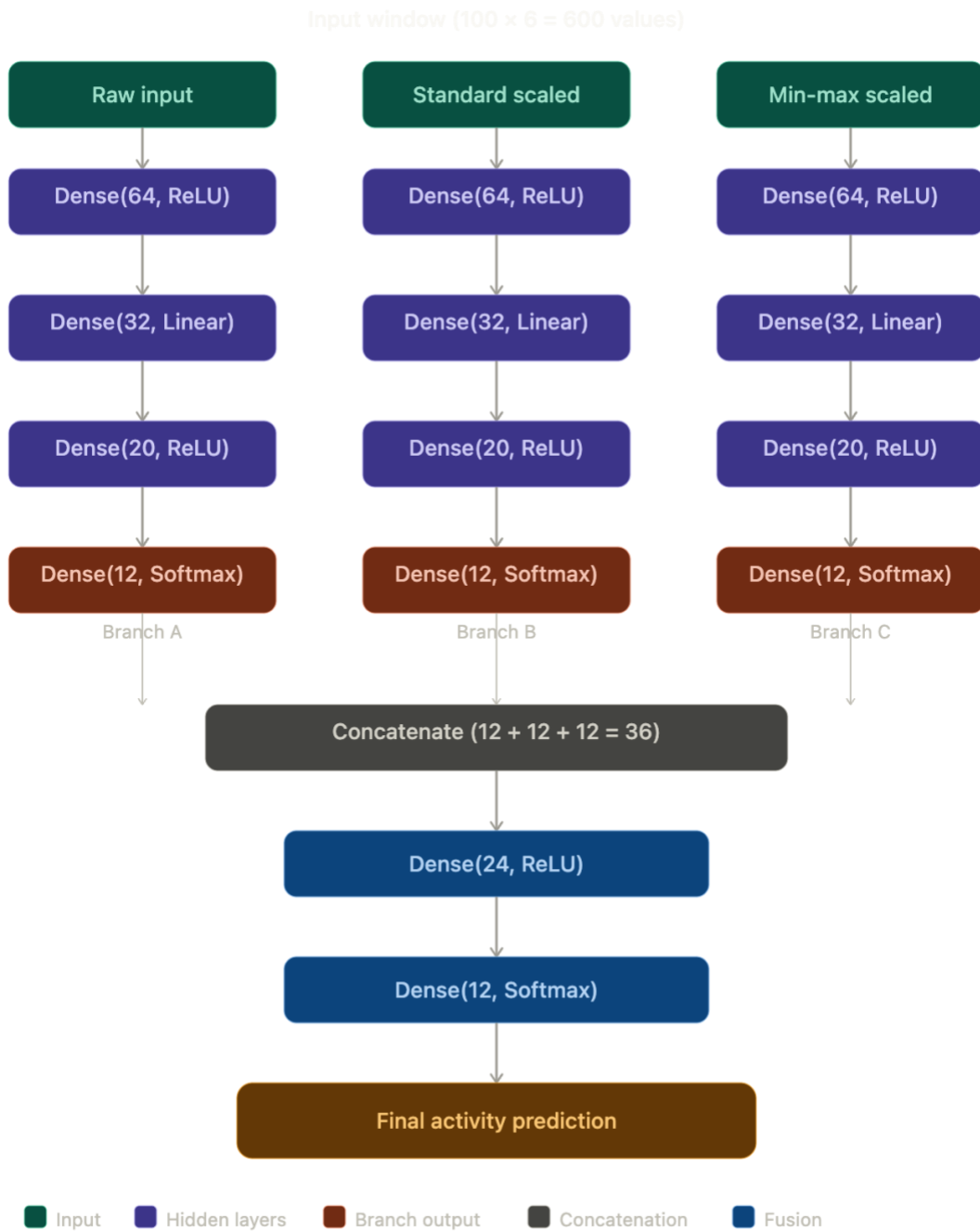
Part 1:

Edge Impulse Link: <https://studio.edgeimpulse.com/public/1017071/live>

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 105 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.000000
  updown: 0.996094
  wave: 0.000000
  anomaly score: 0.260638
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 101 ms., Classification: 5 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.000000
  updown: 0.996094
  wave: 0.000000
  anomaly score: 0.108370
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 105 ms., Classification: 5 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.000000
  updown: 0.996094
  wave: 0.000000
  anomaly score: -0.543974
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 105 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.996094
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.387425
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 105 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.996094
  snake: 0.000000
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.431333
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 105 ms., Classification: 5 ms., Anomaly: 0 ms.):
  idle: 0.996094
  snake: 0.000000
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.431948
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 101 ms., Classification: 5 ms., Anomaly: 1 ms.):
  idle: 0.996094
  snake: 0.000000
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.433688
Starting inferencing in 2 seconds...
█
```

Part 2:

1.



2. The notebook compresses the model using pruning, quantization-aware training (QAT), and INT8 quantization.

Pruning

- Uses polynomial decay pruning.
- Sparsity increases from 10% to 80% over the first 500 training steps.
- Small-magnitude weights are set to zero.

Removing Pruning Wrappers

- Pruning wrappers are stripped using `strip_pruning()`.
- This leaves only the sparse weights and prepares the model for QAT.

Quantization-Aware Training

- QAT simulates INT8 arithmetic during training.
- This helps preserve accuracy after quantization.

Mask Enforcement

- During QAT, pruned weights may become nonzero again.
- The notebook reapplies pruning masks after updates to keep those weights at zero and preserve sparsity.

INT8 TFLite Conversion

- The final model is converted to a fully INT8 TFLite model.
- Representative training data is used to determine activation ranges and quantization parameters.

Benefits for TinyML

- Pruning reduces the number of active weights and memory usage.
- INT8 quantization reduces model size by about 4× and speeds up inference.
- Together they make deployment feasible on the Arduino Nano 33 BLE Sense.

3.

```
Tiny_Ensemble_Learning.ino  encoder_clf_minmax_pruned_qat_int8.cc  encoder_clf_raw_pruned_qat_int8.cc  encoder_clf_std_pruned_qat_int8.cc  stacked_meta_clf_pruned_qat_int8.cc
1 #include "TensorFlowLite.h"

Output Serial Monitor x
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem1101') Both NL & CR 115200 baud

Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9844, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000
Standard-scaled model scores: 0.0000, 0.1133, 0.0195, 0.0000, 0.0039, 0.0000, 0.0547, 0.0078, 0.0000, 0.4609, 0.0000, 0.3398
Min-max-scaled model scores: 0.0000, 0.0000, 0.0156, 0.0000, 0.0586, 0.0000, 0.0000, 0.0000, 0.8945, 0.0195, 0.0078, 0.0000, 0.0000
Meta-classifier scores: 0.0586, 0.3086, 0.0391, 0.0039, 0.0000, 0.0039, 0.0195, 0.0195, 0.0000, 0.0117, 0.0078, 0.5234

Final prediction: Jump front and back
Class index: 11
Confidence score: 0.5234

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9844, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000
Standard-scaled model scores: 0.0000, 0.2305, 0.0508, 0.0000, 0.0039, 0.0000, 0.0391, 0.0156, 0.0000, 0.5117, 0.0000, 0.1523
Min-max-scaled model scores: 0.0039, 0.0000, 0.0664, 0.0000, 0.3164, 0.0000, 0.0000, 0.4961, 0.0625, 0.0508, 0.0000, 0.0000
Meta-classifier scores: 0.0625, 0.2422, 0.1523, 0.0039, 0.0000, 0.0078, 0.0117, 0.0078, 0.0000, 0.0391, 0.0078, 0.4609

Final prediction: Jump front and back
Class index: 11
Confidence score: 0.4609

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9844, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000
Standard-scaled model scores: 0.0000, 0.2539, 0.0703, 0.0000, 0.0039, 0.0000, 0.0469, 0.0195, 0.0000, 0.4648, 0.0000, 0.1406
Min-max-scaled model scores: 0.0039, 0.0000, 0.0430, 0.0000, 0.2344, 0.0000, 0.0039, 0.6328, 0.0312, 0.0547, 0.0000, 0.0000
Meta-classifier scores: 0.0625, 0.2852, 0.1172, 0.0039, 0.0000, 0.0078, 0.0156, 0.0156, 0.0000, 0.0312, 0.0078, 0.4531

Final prediction: Jump front and back
Class index: 11
Confidence score: 0.4531
```

Screenshot: Provided above.

Is the output acceptable? No. The board is stationary yet all three windows confidently predict "Jump front and back" (class 11, scores 0.52 / 0.46 / 0.45). A resting board should not produce a high-confidence dynamic activity label.

Issues observed:

- The raw model scores are nearly identical across all windows (class 2 $\approx$ 0.9844 every time), meaning Branch A contributes no useful diversity to the ensemble.
- The meta-classifier consistently resolves to class 11 regardless of branch disagreement, suggesting a learned bias toward that class.
- Gravity from the flat board ($\sim 1g$ on one axis) is likely being misread as motion if training data was collected in a different orientation.
- A unit or scaling mismatch between training preprocessing and the on-device pipeline would shift all three input branches out of distribution.

Practical improvements:

1. Add an idle class — collect resting data from the deployed board orientation and include it in training.

2. Verify scaler parameters on-device — ensure the exact mean/std and min/max from training are hardcoded in the Arduino sketch and validated against a known input before deployment.
-

Question 4:

Yes — the model locked onto class 11 across all windows regardless of board state. The most likely causes are a training/deployment distribution mismatch (different board orientation or sensor axis mapping means the live gravity vector resembles a training motion pattern), scaler parameter mismatch (out-of-distribution inputs to all three branches), and an overconfident meta-classifier that defaults to its strongest learned association when branch signals are unreliable.